Aritificial Intelligence:

Code:

```
import tkinter as tk
from tkinter import messagebox
import random


# Parking System Class
class ParkingSystem:
    def __init__(self):
        self.slots = {i: False for i in range(1, 51)}  # 50 slots, initially all free
        self.reserved = False  # To track if the user has already reserved a slot

    def check_slot(self, slot_id):
        if 1 <= slot_id <= 50:
            return self.slots[slot_id]
        return None

    def reserve_slot(self, slot_id):
        if self.reserved:
            return "already_reserved"  # If the user has already reserved a slot
        if 1 <= slot_id <= 50:
            if self.slots[slot_id]:
                return "already_reserved"
            else:
                self.slots[slot_id] = True
                self.reserved = True  # Mark the user as having reserved a slot
                return "reserved"
        return "invalid"
```

```
    def free_slot(self, slot_id):
        if 1 <= slot_id <= 50:
            if not self.slots[slot_id]:
                return "already_free"
            else:
                self.slots[slot_id] = False
                self.reserved = False  # Reset reserved status when freeing the slot
                return "freed"
        return "invalid"

    def suggest_next_slot(self):
        """ Simple AI suggestion: If there are reserved slots, suggest a free one near the most recently reserved slot. """
        available_slots = [slot for slot, reserved in self.slots.items() if not reserved]
        return random.choice(available_slots) if available_slots else None

# GUI for Parking System
class ParkingApp:
    def __init__(self, root, parking_system):
        self.root = root
        self.parking_system = parking_system
        self.create_widgets()

    def create_widgets(self):
        self.slot_id_label = tk.Label(self.root, text="Enter Slot ID (1-50):")
        self.slot_id_label.pack(pady=10)

        self.slot_id_entry = tk.Entry(self.root)
```

```
        self.slot_id_entry.pack(pady=5)

        self.check_button = tk.Button(self.root, text="Check Slot", command=self.check_slot)
        self.check_button.pack(pady=5)

        self.reserve_button = tk.Button(self.root, text="Reserve Slot", command=self.reserve_slot)
        self.reserve_button.pack(pady=5)

        self.free_button = tk.Button(self.root, text="Free Slot", command=self.free_slot)
        self.free_button.pack(pady=5)

        self.suggest_button = tk.Button(self.root, text="Suggest Next Slot", command=self.suggest_slot)
        self.suggest_button.pack(pady=5)

        self.status_label = tk.Label(self.root, text="Status: ")
        self.status_label.pack(pady=10)

    def check_slot(self):
        try:
            slot_id = int(self.slot_id_entry.get())
            status = self.parking_system.check_slot(slot_id)
            if status is None:
                self.status_label.config(text="Invalid Slot ID!")
            elif status:
                self.status_label.config(text=f"Slot {slot_id} is RESERVED.")
            else:
                self.status_label.config(text=f"Slot {slot_id} is FREE.")
        except ValueError:
            messagebox.showerror("Input Error", "Please enter a valid slot ID between 1 and 50.")
```

```
def reserve_slot(self):
    try:
        slot_id = int(self.slot_id_entry.get())
        result = self.parking_system.reserve_slot(slot_id)
        if result == "reserved":
            self.status_label.config(text=f"Slot {slot_id} has been RESERVED.")
        elif result == "already_reserved":
            self.status_label.config(text="Sorry, you can not reserve more than one slot.")
        elif result == "invalid":
            self.status_label.config(text="Invalid Slot ID!")
    except ValueError:
        messagebox.showerror("Input Error", "Please enter a valid slot ID.")

def free_slot(self):
    try:
        slot_id = int(self.slot_id_entry.get())
        result = self.parking_system.free_slot(slot_id)
        if result == "freed":
            self.status_label.config(text=f"Slot {slot_id} has been FREED.")
        elif result == "already_free":
            self.status_label.config(text=f"Slot {slot_id} is already FREE.")
        elif result == "invalid":
            self.status_label.config(text="Invalid Slot ID!")
    except ValueError:
        messagebox.showerror("Input Error", "Please enter a valid slot ID.")

def suggest_slot(self):
    suggested_slot = self.parking_system.suggest_next_slot()
```

```
        if suggested_slot:
            self.status_label.config(text=f"AI suggests parking slot: {suggested_slot}")
        else:
            self.status_label.config(text="No available slots based on AI prediction.")

# Main function to run the app
def main():
    root = tk.Tk()
    root.title("Parking System")

    parking_system = ParkingSystem()  # Instantiate the ParkingSystem here
    app = ParkingApp(root, parking_system)  # Pass the system to the ParkingApp

    root.mainloop()

if __name__ == "__main__":
    main()
```